

Git Command Reference

Setup · Daily Workflow · Branching · Versioning · Recovery

1 Initial Setup

Run once per machine (stored in ~/.gitconfig).

<code>git config --global user.name "Name"</code>	Set your display name on all commits
<code>git config --global user.email "a@b.com"</code>	Set your email address on all commits
<code>git config --global init.defaultBranch main</code>	Use 'main' instead of 'master' by default
<code>git config --global core.autocrlf true</code>	Windows: auto-convert line endings (CRLF<->LF)
<code>git config --list</code>	View all current config settings

2 Repository Creation

<code>git init</code>	Create a new empty repo in the current folder
<code>git init -b main</code>	Create new repo with 'main' as the default branch
<code>git clone</code>	Copy a remote repo to your local machine
<code>git clone my-folder</code>	Clone into a specific folder name
<code>git remote add origin</code>	Link local repo to a remote (GitHub, etc.)
<code>git remote -v</code>	List all configured remote URLs
<code>git remote set-url origin</code>	Change the remote URL

3 Daily Workflow — Stage, Commit, Push

<code>git status</code>	Show what's changed, staged, or untracked
<code>git diff</code>	Show unstaged changes (line by line)
<code>git diff --staged</code>	Show changes already staged for commit
<code>git add</code>	Stage a specific file
<code>git add -A</code>	Stage all changes (new + modified + deleted)

<code>git add .</code>	Stage all changes in current directory
<code>git commit -m "message"</code>	Commit staged changes with a message
<code>git commit -am "message"</code>	Stage tracked files + commit in one step
<code>git push</code>	Push current branch to remote
<code>git push -u origin main</code>	Push and set upstream (first time)
<code>git pull</code>	Fetch + merge remote changes
<code>git pull --rebase</code>	Fetch + rebase instead of merge (cleaner history)
<code>git fetch</code>	Download remote changes without merging

Commit message convention: type(scope): short description

<code>feat: add logistic regression model</code>	New feature
<code>fix: remove deprecated multi_class arg</code>	Bug fix
<code>refactor: split data loader into utils</code>	Code restructure, no feature change
<code>docs: update README with run commands</code>	Documentation only
<code>chore: add .gitignore for data/</code>	Maintenance / tooling

4 Branching & Merging

<code>git branch</code>	List all local branches
<code>git branch -a</code>	List local + remote branches
<code>git branch feature/my-feature</code>	Create a new branch (stay on current)
<code>git checkout -b feature/my-feature</code>	Create and switch to a new branch
<code>git switch main</code>	Switch to an existing branch (modern syntax)
<code>git merge feature/my-feature</code>	Merge branch into current branch
<code>git merge --no-ff feature/x</code>	Merge with a merge commit (preserves history)
<code>git rebase main</code>	Replay current branch on top of main
<code>git branch -d feature/my-feature</code>	Delete a branch (safe — blocks if unmerged)
<code>git branch -D feature/my-feature</code>	Force-delete a branch
<code>git push origin --delete feature/x</code>	Delete a remote branch

5 Versioning & Tags

Tags mark specific commits as release milestones (e.g. v1.0, v2.3.1).

<code>git tag</code>	List all tags
<code>git tag v1.0</code>	Create a lightweight tag at HEAD
<code>git tag -a v1.0 -m "Stable release"</code>	Create an annotated tag with message
<code>git tag -a v1.0 abc123 -m "msg"</code>	Tag a specific commit by hash
<code>git push origin v1.0</code>	Push a single tag to remote
<code>git push --tags</code>	Push ALL tags to remote
<code>git tag -d v1.0</code>	Delete a local tag
<code>git push origin --delete v1.0</code>	Delete a remote tag
<code>git checkout v1.0</code>	Check out code at a specific tag (detached HEAD)
<code>git describe --tags</code>	Show nearest tag + commits since it

Semantic Versioning: MAJOR.MINOR.PATCH (e.g. v2.1.4)

MAJOR (v2.0.0)	Breaking change — incompatible API
MINOR (v1.3.0)	New feature — backward compatible
PATCH (v1.2.5)	Bug fix — backward compatible

6 Inspection & History

<code>git log</code>	Full commit history
<code>git log --oneline</code>	Compact one-line history
<code>git log --oneline --graph --all</code>	Visual branch graph
<code>git log -n 10</code>	Show last 10 commits only
<code>git log --</code>	History for a specific file
<code>git show abc123</code>	Show diff + metadata for a commit
<code>git blame</code>	Who changed each line and when
<code>git diff main..feature/x</code>	Compare two branches
<code>git stash list</code>	List all saved stashes
<code>git shortlog -sn</code>	Commit count per author

7 Undoing Changes

<code>git restore</code>	Discard unstaged changes to a file
<code>git restore --staged</code>	Unstage a file (keep changes in working dir)

<code>git checkout --</code>	Older syntax for restoring a file
<code>git revert abc123</code>	Create a new commit that undoes a past commit
<code>git reset --soft HEAD~1</code>	Undo last commit — keep changes staged
<code>git reset --mixed HEAD~1</code>	Undo last commit — keep changes unstaged
<code>git reset --hard HEAD~1</code>	Undo last commit — DISCARD all changes
<code>git clean -fd</code>	Delete all untracked files & directories
<code>git commit --amend -m "msg"</code>	Rewrite the last commit message

Warning: reset --hard and clean -fd are destructive and cannot be undone. Use git stash to save work temporarily before resetting.

8 Stashing — Save Work Without Committing

<code>git stash</code>	Save all uncommitted changes to a stash
<code>git stash push -m "desc"</code>	Stash with a description
<code>git stash pop</code>	Restore most recent stash + remove it
<code>git stash apply stash@{1}</code>	Apply a specific stash (keep it in the list)
<code>git stash drop stash@{0}</code>	Delete a specific stash
<code>git stash clear</code>	Delete ALL stashes
<code>git stash branch feature/x</code>	Create a new branch from a stash

9 DSF504 Project Workflow (run from PowerShell)

Git must be run from Windows PowerShell — not from the Cowork bash sandbox.

First-time setup

<code>cd C:\DSF504</code>	Navigate to project root
<code>git init -b main</code>	Initialise repo
<code>git config user.name "Ko-Yang"</code>	Set identity
<code>git config user.email "kyw@fusions360.com"</code>	Set email
<code>git add -A</code>	Stage all project files
<code>git commit -m "feat: initial DSF504 ML platform"</code>	First commit
<code>gh repo create dsf504-ml-platform --public --source=. --remote=origin --push</code>	Create GitHub repo + push

After every work session

<code>git add -A</code>	Stage all changes
<code>git commit -m "fix: describe change"</code>	Commit with a meaningful message
<code>git push</code>	Push to GitHub

Before a big change (e.g. editing app.py)

<code>git stash</code>	Save current state
<code># ... make changes ...</code>	
<code>git stash pop</code>	Restore saved state if needed

Tag a stable release

<code>git tag -a v1.0 -m "Stable: all 5 use cases complete"</code>	Create annotated tag
<code>git push --tags</code>	Push tag to GitHub

If .git becomes corrupt (from a failed sandbox attempt)

<code>Remove-Item -Recurse -Force C:\DSF504\.git</code>	Delete corrupt .git (PowerShell)
<code>git init -b main</code>	Re-initialise
<code>git remote add origin</code>	Re-link remote
<code>git add -A && git commit -m "restore"</code>	Re-commit
<code>git push --force-with-lease</code>	Push (only if remote is in sync)